

《信息系统安全》第十四讲

数据库安全

Part II 数据库安全的一般性原理

2023年12月18日

周亚建

yajian@bupt.edu.cn

Overview of Last Class

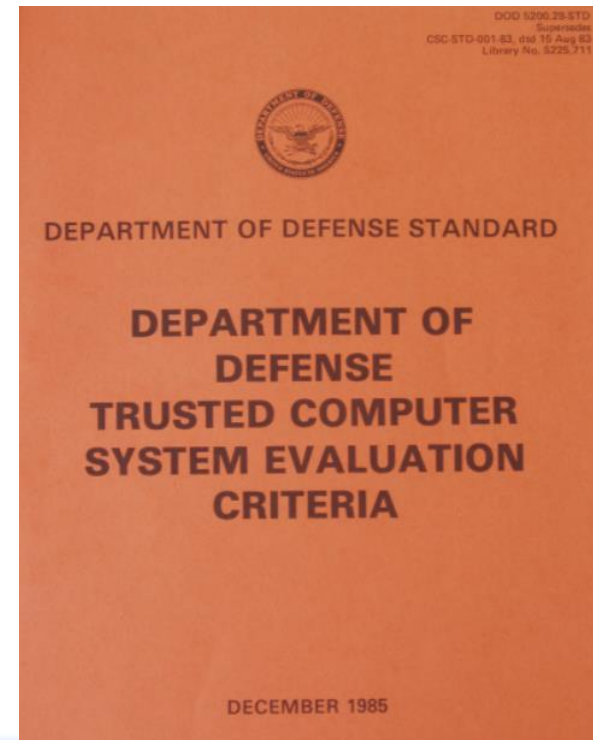
数据库为什么存在安全问题？

如何应对数据库的漏洞？

数据库安全和操作系统安全有无关联？

安全相关标准

1. **《可信计算机系统评估准则》 TCSEC——桔皮书和《可信计算机系统评估标准关于可信数据库的解释》 TDI**
2. **Information Technology Security Common Criteria
通用安全评价准则 CC**
3. **计算机信息系统安全保护等级划分准则：
国标17859-1999**
4. **中国推荐标准GB/T18336-2001**



可信计算机系统评估准则— TCSEC及其在数据库管理系统中的解释TDI

安全等级	定义
A1	验证设计（ Verified Design ）
B3	安全域（ Security Domains ）
B2	结构化保护（ Structural Protection ）
B1	标记安全保护（ Labeled Security Protection ）
C2	受控的存取保护（ Controlled Access Protection ）
C1	自主安全保护 （ Discretionary Security Protection ）
D	最小保护（ Minimal Protection ）

可信计算机系统评估准则— TCSEC及其在数据库管理系统中的解释TDI

可信计算基 (Trusted computing base)：系统的安全依赖于具体实施安全策略的可信软件和硬件。这些软件、硬件和负责系统安全管理的人员一起组成了TCB。

数据库系统的TCB组成

- 操作系统安全内核
- 数据库管理系统内核或者实施安全管理的部件
- 处理敏感信息的程序
- 实施安全策略有关的文件
- 其他有关的固件、硬件和设备
- 负责系统管理的人员
- 保障固件和硬件正确的程序和诊断软件

可信计算机系统评估准则— TCSEC及其在数据库管理系统中的解释TDI

D级： 最低保护

D级不需要安全特性。

C1级： 自主安全保护

能够实现对用户和数据的分离，具备自主访问控制，保护或限制用户传播权限的能力。

C2级： 受控访问保护

- 自主访问控制的粒度更细；
- 审计踪迹能够追踪到每个用户对每个对象的访问

可信计算机系统评估准则— TCSEC及其在数据库管理系统中的解释TDI

C2级：受控访问保护——从4个方面衡量

- (1) 安全策略——主体访问控制、客体重用
- (2) 责任——标识与鉴别、审计
- (3) 保证——操作性保证、生命周期保证
- (4) 文档——安全特性用户指南、可信设施手册、测试文档

可信计算机系统评估准则—— TCSEC及其在数据库管理系统中的解释TDI

B级：强制保护：定义并保持敏感度标记的完整性，实施基于标记的强制访问控制

B1级：标记安全保护

- (1) 安全策略——自主访问控制、客体重用、标记、强制访问控制
- (2) 责任——标识与鉴别、审计在C2级的基础上有所增强
- (3) 保证——操作性保证、生命周期保证
- (4) 文档——安全特性文档、可信设施手册、测试文档、设计文档

可信计算机系统评估准则— TCSEC及其在数据库管理系统中的解释TDI

B2级：结构化保护级 和B1级的区别：

- (1) 形式化的安全模型并对系统内所有的主体和客体实施MAC和DAC
- (2) 存储隐通道分析

B3级：安全域

- ❖ 安全功能必须足够小
- ❖ 系统审计设施能够识别何时将发生入侵
- ❖ 设计与实现的一致性验证

国际通用安全评价准则CC

简介和一般模型

(1) CC开发的目的是应用范围

- ❖ 使各种安全评估结果具有可比性
- ❖ 在安全性评估过程中为信息系统及其产品的安全功能和保证措施提供一组通用要求，确定一个可信级别

(2) 组成

- ❖ 简介和模型——评估通用模型
- ❖ 安全功能要求
- ❖ 安全保证要求——保证组件和保证级别

(3) CC 中的缩略语及常用术语——EAL、TOE、PP、TSP、SF、ST、TSF

国际通用安全评价准则CC

CC的评估保障等级（EAL）有7层

保障等级的升高完全依赖于在同一保障族中替换更高层次的保障构件，并且从其它保障族中增加保障构件。

EAL1—功能测试

EAL2 —结构测试

EAL3 —方法测试和检验（check）

EAL1是入门级，1~4级
不断扩充细节和内容，
但没有引入专门的
重要安全机制

EAL4 —方法设计，测试和评审（review）

EAL5 —半正式设计和测试

EAL6 —半正式验证（verify）的设计和测试

EAL7 —正式验证的设计和测试

需要专门的安全
工程技术，并一
步步增强

中国国家标准17859-1999

将计算机系统安全保护能力分为下面5个等级

第一级 用户自主保护级

自主访问控制、身份鉴别、数据完整性

第二级 系统审计保护级

除扩充上述内容外，还包括客体重用、审计

第三级 安全标记保护级

除扩充上述内容外，还包括强制访问控制、标记；

第四级 结构化保护级

除扩充上述内容外，还包括隐蔽信道分析、可信路径；

第五级 访问验证保护级

除扩充上述内容外，还包括可信恢复；

计算机系统安全等级评估标准一览表

Trusted Computer System Evaluation Criteria <i>美国</i> TCSEC	Information Technology Security Evaluation Criteria <i>欧洲</i> ITSEC	Canada Trusted Computer Product Evaluation Criteria <i>加拿大</i> CTCPEC	Information Technology Security Common Criteria CC 标准	
D:最小保护	E0	T0	——	
——	——	T1	EAL1—功能测试	
C1:任意安全保护	E1	T2	EAL2—结构测试	1:用户自主保护级
C2:控制存取保护	E2	T3	EAL3—方法测试和检验	2:系统审计保护级
B1:标记安全保护	E3	T4	EAL4—方法设计, 测试和评审	3:安全标记保护级
B2:结构保护	E4	T5	EAL5—一半正式设计和测试	4:结构化保护级
B3:安全域	E6	T6	EAL6—一半正式验证的设计和测试	5:访问验证保护级
A1:验证设计	E7	T7	EAL7—正式验证的设计和测试	



数据库管理系统的访问控制

数据库系统的访问控制

访问控制

基本要求：

- (1) 保证只授权给有资格的用户访问数据库中数据的权限；
- (2) 所有未正常授权的人员均无法接近数据，防止和杜绝非授权的数据访问。

数据库系统的要求：

- (1) 对更加精细的数据粒度加以控制：表、视图、元组、列、元素（每个元组的字段）等。
- (2) 定义完整的访问操作
- (3) 对访问规则进行检查

数据库管理系统自主访问控制

关系的基本概念

数据库中的基本对象是关系，关系是 n -元组的集合。
每个 n -元组有 n 个命名的列

employee(name, salary, manager, department)

两种关系：

(1) 物理存储数据的**基表**

(2) 虚关系——**视图**

关系的基本概念——视图

视图是基表上的窗口

关系的行和列的子集；关系内信息的概要；两个或多个关系的连接

Create view vemp(name, salary)

as

select name, salary from employee where department='toy'

Create view avgsal(avg_salary)

as

select avg(salary) from employee group by department

Create view locemp(name, floor)

as

**select name, floor from employee, dept
where employee.department = dept.department.**

关系的基本概念——DBMS中的自主访问控制模型及假设

访问控制模型

- ① 多数数据库管理系统采用的是**访问控制列表（或访问控制矩阵）**的授权模式，允许权限的授予与回收
- ② 视图机制是传统的数据库实现行级或列的子集的授权的方法

前提假设：

- ① 用户已经被正确地标识，并且登陆进入数据库
- ② 这个用户没有伪造、借用、盗取其他用户的鉴别信息

SQL中的授权——GRANT命令

初始权限的拥有者：

- ❖ 数据库管理员（DBA）
- ❖ 数据库对象的创建者

SQL命令：

GRANT ALL PRIVILEGES | privileges ON <table> TO <user-list> [WITH GRANT OPTION]

例：A是EMPLOYEE的创建者，假设他发出命令：

GRANT SELECT, INSERT ON EMPLOYEE TO B;

B用户发出的对EMPLOYEE的全表查询语句是怎样的？

SELECT * FROM A.EMPLOYEE;

SQL中的授权——GRANT命令

例：

**A: GRANT SELECT, INSERT ON EMPLOYEE TO B
WITH GRANT OPTION**

**A: GRANT SELECT ON EMPLOYEE TO X
WITH GRANT OPTION**

B: GRANT SELECT, INSERT ON **A.EMPLOYEE TO X;**

DBMS将接收者的权限分两类：可转授的和不可转授的。
除非用户被授予权限**P**的转授权，否则该用户不能将
权限**P**授予他人。（**WITH GRANT OPTION实现**）

DBMS的访问控制机制

自主访问控制需要达到的目标

- ① 用户可以保护自身拥有的信息;
- ② 对象的拥有者可以将对象的访问权限授予其他用户;
- ③ 对象的拥有者可以指定给其他用户的访问权限及其转授权限。

——分两种情况讨论：表 and 视图

DBMS的访问控制机制

表级权限的授予与回收的访问控制规则

- ① 表的创建者具备表上的一切权限（增、删、查、改）以及这些权限的授出权限，授出权限时，还可以指定这些授出权限的转授权；
- ② 允许同一用户对同一权限接收者在同一对象上重复授予相同的授权；
- ③ 权限的授予者可以指定权限的转授权；
- ④ 不允许授权构成环路（有些DBMS允许）；
- ⑤ 权限的级联式回收。

访问控制矩阵模型

	Object 1	Object 2	...	Object j	...
Subject 1	read, write	own, read, write	...	read	...
Subject 2	read	write	...	own, read, write	...
...	...	...	...	...	...
Subject i	own, read, write	read	...	read, write	...
...	...	...	...	...	...

图 2 访问矩阵模型

如图 2 所示，访问矩阵是以主体为行索引、以客体为列索引的矩阵，矩阵中

DBMS的访问控制机制

自主访问控制的设计

问题：若要实现授权语句要考虑哪些问题？怎样设计数据结构？

假设：

- (1) 不考虑授权语句的语句格式及语法分析；
- (2) 假设有关的数据结构可以以表的形式存放在数据库中。

权限的授予在DBMS中的一种实现

(1) 权限的存储——系统表记录权限的授予信息

表级权限和列级权限分开存储，用系统表 **SYSAUTH** 和 **SYSCOLAUTH** 分别存放表级和列级权限

SYSAUTH记录的信息：（注意存储和内存数据结构不同）

USERID	指定被授权的用户，即权限的接收者
GRANTOR	权限的授予者的ID
TNAME	指定表名
TYPE	表示授权的是基表还是视图
PRIVILEGES	记载表上的每一种权限(表级权限)
UPDATE	指定列上的UPDATE权限（ALL, SOME, NONE）
GRANTOPT	权限是否可以转授给他人
Timestamp	授权时间戳（？）

权限的授予在DBMS中的一种实现

(2) 权限的授予流程

- ① 用户发出GRANT命令;
- ② 授权模块分析决定这个权限是否被授予; (什么情况下允许授权?) 用户是1. DBA; 2.数据库对象的创建者; 3.拥有该权限以及转授权限的用户
- ③ 执行GRANT命令, 其结果就是往SYSAUTH中插入一条新的元组。
- ④ 若出现重复授权, 则不会在这两个系统表中产生重复的记录, 只记录最早授权的时间戳。
- ⑤ 若不允许授权环路的出现则做授权环路检查。

是否只要考虑权限授予即可? 还需要做什么吗?

DBMS的访问控制机制

视图上的权限授予

视图的情况，怎么办？

——比较复杂，因为视图还有其所基于的基表和视图，怎样处理？

DBMS的访问控制机制——视图上的权限授予功能

定义视图时：

- ① 用户在视图上的权限是由视图所在基表上的权限决定的，若视图包含不止一个基表，则用户的视图权限是他在这些基表上的权限的交集；
- ② 用户在所定义视图上的权限P是可转授的当且仅当视图在其基表上有可转授的权限P。——这是一种做法，另一种实现方法是视图的创建者可以将视图的访问权限任意授予他人；
- ③ 具备在视图的列上的更新权限仅当具备出现在视图中的所有列上的更新权限时；
- ④ 当一个用户接收一个视图的权限，对视图进行访问时，有两种处理方法：
 - 以该视图创建者的身份去访问视图
 - 检查当前用户在视图所基于的表集合上的权限

权限的回收

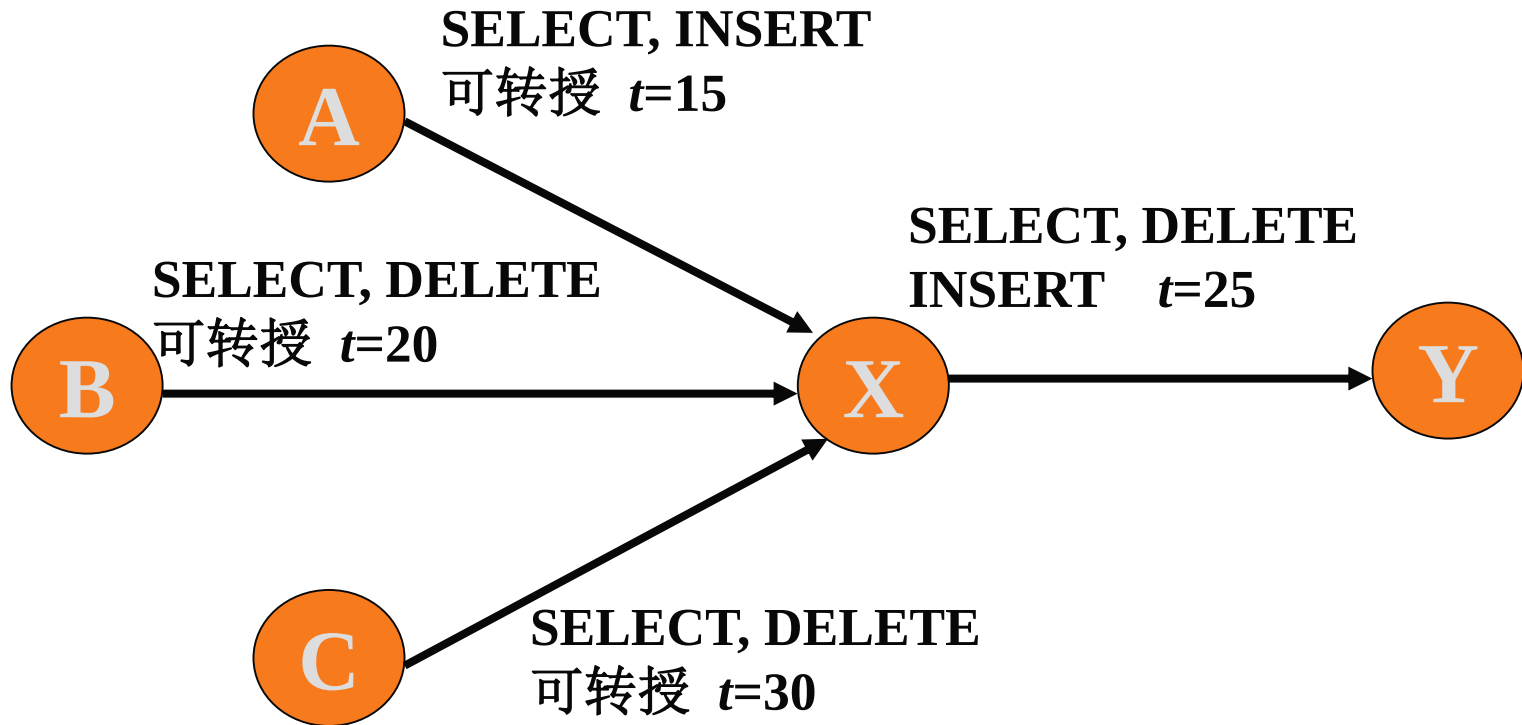
(1) 回收权限的SQL命令

```
REVOKE ALL PRIVILEGES | privileges on  
<table> FROM <user-list>
```

(2) 权限回收规则

- ❖ 在指定的表上的指定权限被否定，除非被回收权限的用户有另外的权限来源；
- ❖ 对于回收，系统必须（递归地）从指定授权表的接收者处回收权限（级联式回收）；
- ❖ 若被回收权限者回收权限之后仍然拥有被回收的权限，而这些权限来源于其他用户，则他仍然保留这些权限。

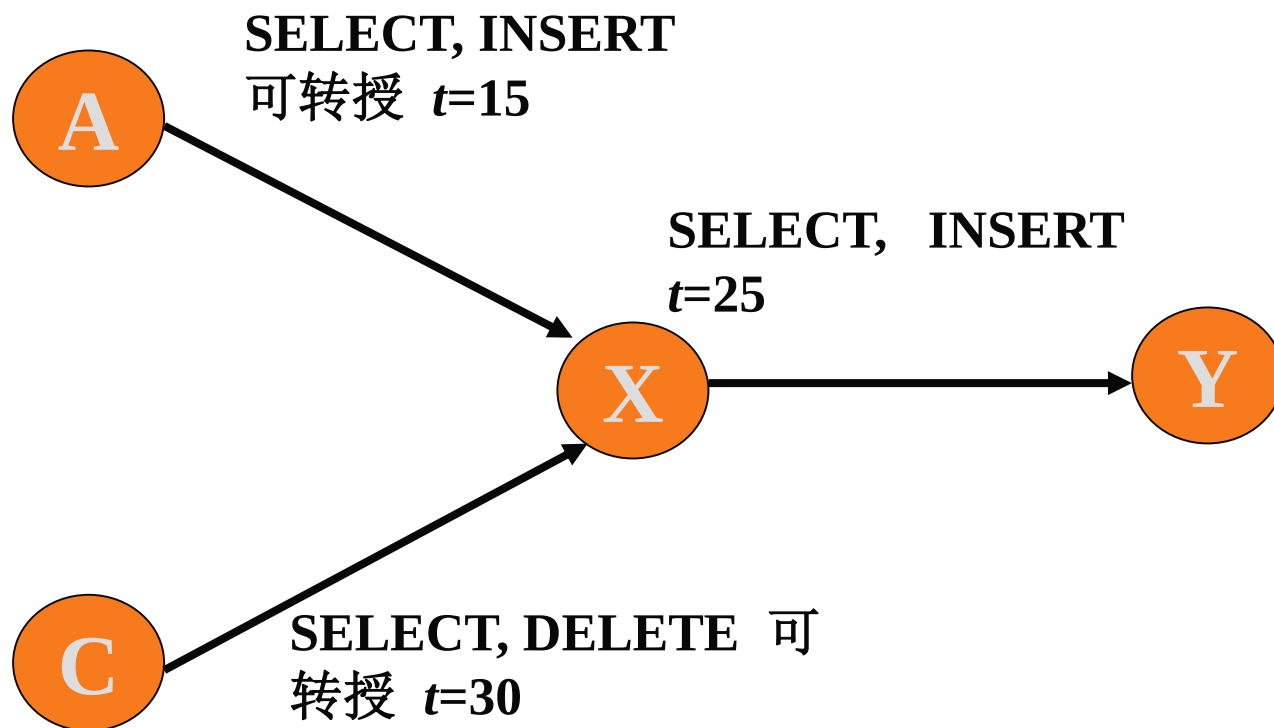
权限的级联式回收



假设在时间 $t=35$, B发出命令**REVOKE ALL PRIVILEGES ON EMPLOYEE FROM X**

则: X在时间 $t=25$ 时的**DELETE**权限的授予必须回收, 而X的**SELECT**和**INSERT**授权被允许保留, 因为它们仍然被早些时候授予的权限支持。

权限的级联式回收



回收权限后的结果

什么时候该回收？ 什么时候不该回收？

表级权限的级联式回收

USERID	TABLE	GRANTOR	SELECT	INSERT	DELETE
X	EMPLOYEE	A	15	15	0
X	EMPLOYEE	B	20	0	20
Y	EMPLOYEE	X	25	25	25
X	EMPLOYEE	C	30	0	30

执行REVOKE之后X的剩余权限 X在时间 $t=25$ 时的DELETE权限的授予必须回收，
TABLE SELECT 因为X的最早DELETE权限是在 $t=30$ 接收的。

EMPLOYEE	{15, 30}	{15}	{30}
X 的授出权限列表:			
TABLE	SELECT	INSERT	DELETE
EMPLOYEE	{25}	{25}	{25}

表级权限的级联式回收

```
REVOKE(grantee, privilege, table, grantor);
/*对于权限<privilege>关闭从<grantor>获得的权限接收者的权限*/
set (privilege) = 0 in the (grantee, privilege, table, grantor) tuple in SYSAUTH;
/*对于权限接收者在表<table>上的剩余可转授<privilege>中找出最小时间戳*/
m ← current timestamp;
for each grantor u such that (grantee, privilege, table, u, grantable) is in
SYSAUTH do
    if privilege != 0 and timestamp < m
        then m ← timestamp;
/* 回收接收者在表上的、在时间m之前授出的权限*/
for each user u' such that (u', privilege, table, grantee) is in SYSAUTH do
    if timestamp < m /*授出权限的时间戳小于剩余权限的时间戳，级联回收*/
        then REVOKE(u', privilege, table, grantee);
return
end REVOKE
```

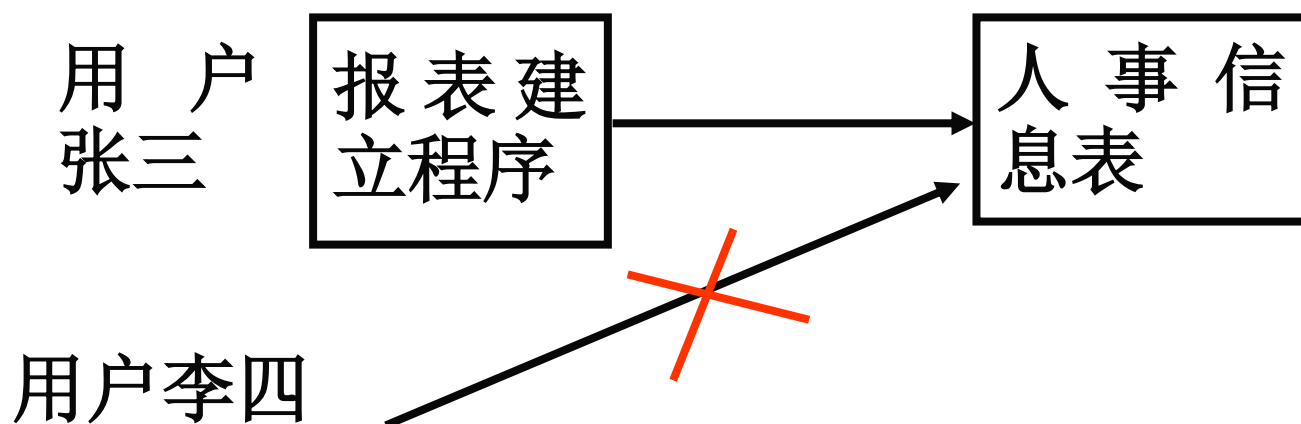
自主访问控制存在的问题

特洛伊木马：一种软件或者程序，隐藏在授权用户所使用的程序中，在满足用户正常功能、在用户毫不知情的情况下，偷偷地做数据泄露的勾当，从而导致安全漏洞。可见这些都不是正常用户所期望的

特洛伊木马可以隐藏在硬件、固件中，还有可能隐藏在一些关键的软件中，如操作系统、编译程序、数据库管理系统中。

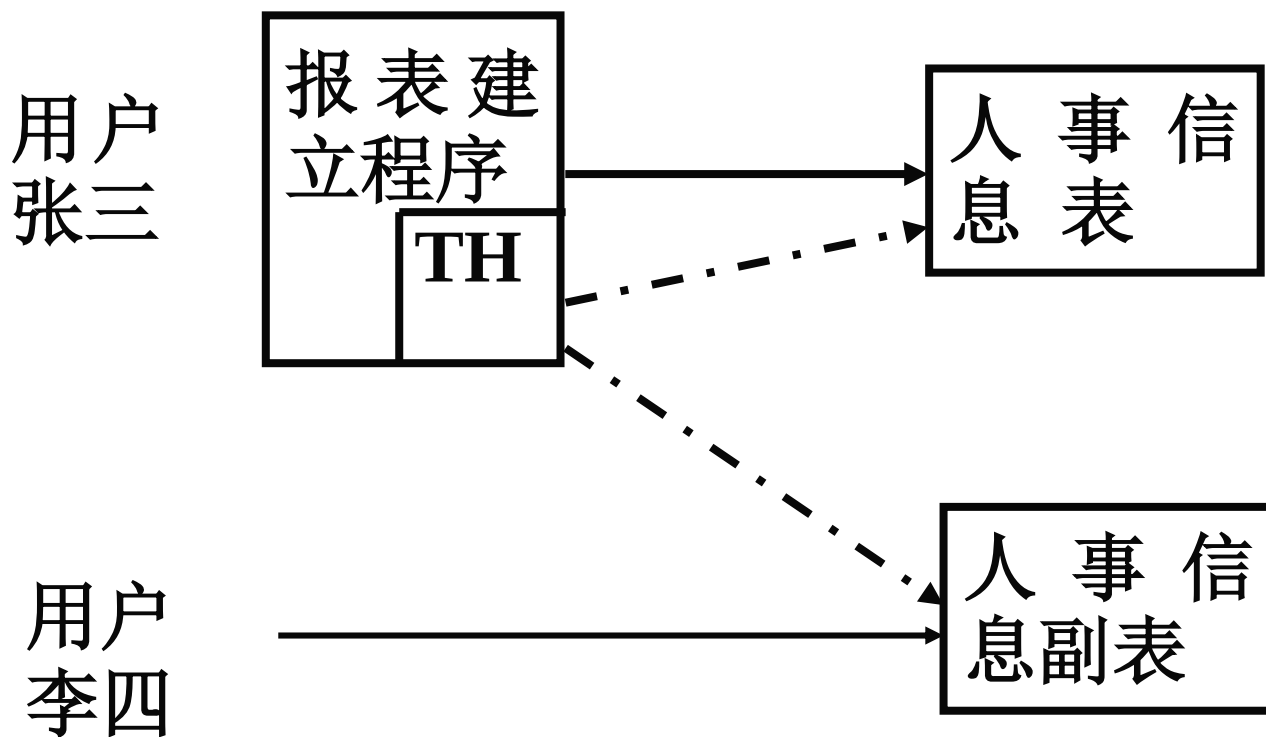
讨论：自主访问控制存在怎样的安全漏洞，构造一个场景。若一个用户U1有表t1的查询权限，但是无t1的查询转授权限，怎样把t1中的信息告诉无t1访问权限的用户U2？假设U1还有其他权限。

自主访问控制存在的问题



正常情况下DBMS拒绝李四对人事信息表的访问

自主访问控制存在的问题



自主访问控制不能抵抗特洛伊木马的攻击

数据库管理系统的强制访问控制

强制访问控制 ——为什么要引入强制访问控制

给张三指定一个访问许可：机密，即张三最多只能访问安全级是机密的数据

李四的访问许可是：普通

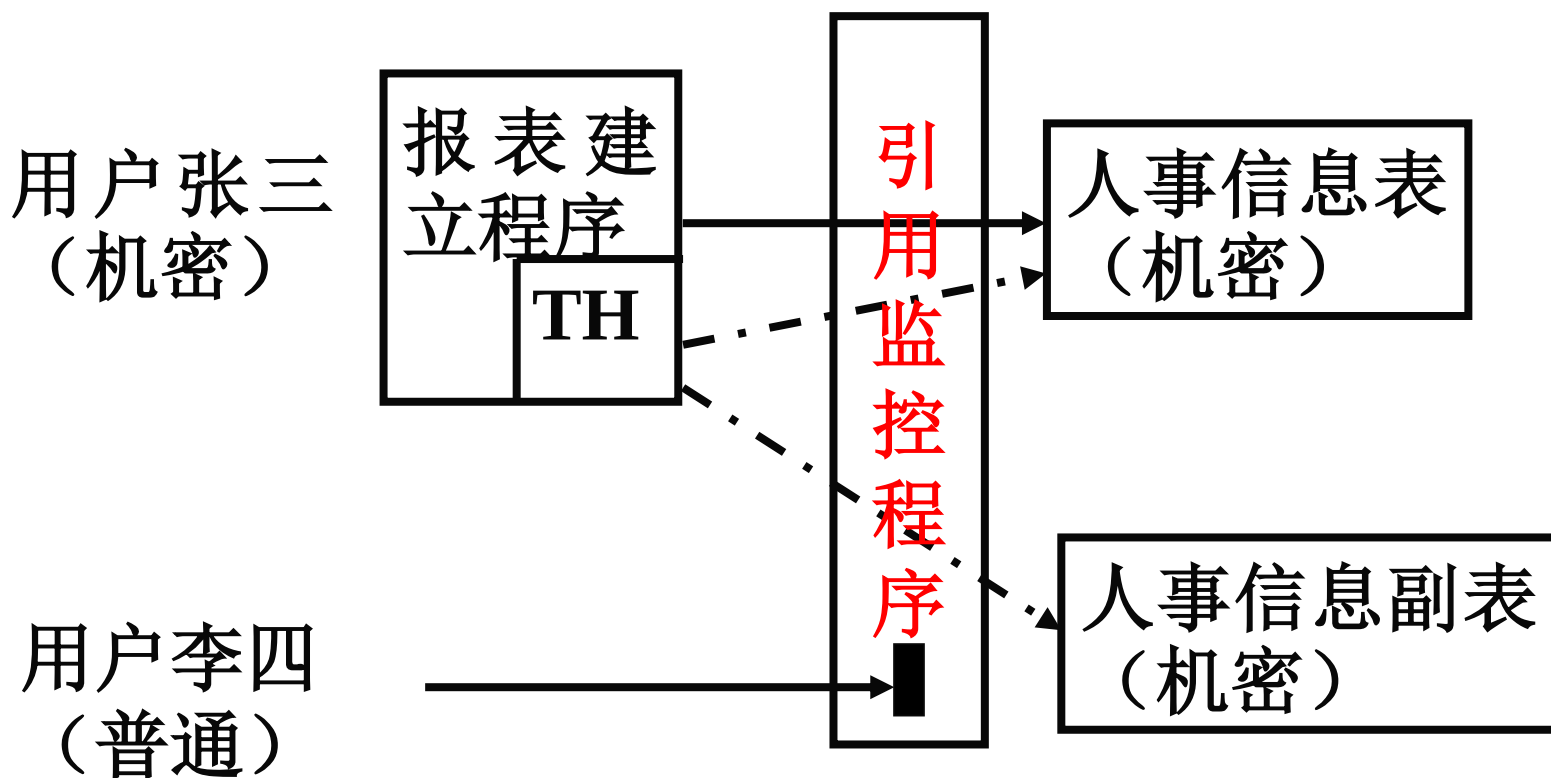
给人事信息表一个安全级：机密

系统访问规则：凡是用户新创建的表的安全级等于该用户当前的访问许可。

系统设立访问检查机制——引用监控机制，用于检查用户的安全许可与所访问对象安全级之间是否匹配：

- 机密级别的用户可以访问普通级别和机密级别的数据；
- 普通级别的用户只能访问普通级别的数据，不能访问机密级别的数据

为什么要引入强制访问控制



MAC对数据本身进行密级标记，无论数据如何复制，标记与数据是一个不可分的整体，只有符合密级标记要求的用户才可以操纵数据。

***** MAC 的主要作用就是防止特洛伊木马的威胁^[1]

为什么要引入强制访问控制

强制访问控制——BLP模型的基本元素

主体 S : 是一个主动的进程或者代表访问数据库的用户的任务/事务，可请求对数据库中信息的访问使信息流动的实体。

客体 O : 是一种被动的实体，如数据库、表、记录和元素等。

安全级: (密级, 范围) 组成的二元组

数据库管理系统的强制访问控制

强制访问控制——BLP模型的基本元素

密级：是一个集合，该集合中的元素是有序可比的，如：普通 < 秘密 < 机密 < 绝密 —— 用 C 表示

范围：是非分层元素的一个子集，如：{计算机系，自控系，光学系} 等 —— 用 K 表示

安全级的例子：（秘密，{计算机系，自控系}）

安全级的比较：若 L_1 、 L_2 分别代表两个安全级，且 $L_1 = (C_1, S_1)$, $L_2 = (C_2, S_2)$ ，则 $L_1 \geq L_2$ 当且仅当 $C_1 \geq C_2$ ，且 $S_1 \supseteq S_2$ 时。

每一个主体都被指定一个**访问许可（安全级）**，每一个客体都指定一个安全级。

数据库管理系统的强制访问控制

强制访问控制——BLP模型的基本元素

访问属性集： $A=\{r, w, e, a, c\}$ ， 其中：

r： 只读；

w： 读写；

e： 执行；

a： 添加（只写）；

c： 控制。

访问矩阵集： $\mu=\{M_1, M_2, \dots, M_p\}$ ， 其中元素 M_k 是一 $n \times m$ 的矩阵， M_k 的元素 M_{ij} 是集合 A 的子集。

数据库管理系统的强制访问控制

强制访问控制——BLP模型的基本元素

请求元素集： $RA=\{g, r, c, d\}$

g: get（得到）， give（赋予）

r: release（释放）， rescind（撤销）

c: change（改变客体的安全级）， create（创建客体）

d: delete（删除客体）

判断集（结果集）： $D=\{yes, no, error, ?\}$ ， 其中

yes: 请求被执行；

no: 请求被拒绝；

error: 系统出错， 有多个规则适合于这一请求；

?: 请求出错， 规则不适用于这一请求。

数据库管理系统的强制访问控制

强制访问控制——BLP模型的基本元素

$F = C^S \times C^O \times (P_K)^S \times (P_K)^O$, 其中:

$C^S = \{f_1 \mid f_1: S \rightarrow C\}$	f_1 : 主体的密级;
$C^O = \{f_2 \mid f_2: O \rightarrow C\}$	f_2 : 客体的密级;
$(P_K)^S = \{f_3 \mid f_3: S \rightarrow P_K\}$	f_3 : 主体的范围集;
$(P_K)^O = \{f_4 \mid f_4: O \rightarrow P_K\}$	f_4 : 客体的范围集。

其中, P_K 表示 K 的幂集。 F 的元素记作 $f = (f_1, f_2, f_3, f_4)$,

给出在某状态下每一主体的密级和范围集, 每一客体的密级和范围集。

- 主体的许可证级 (f_1, f_3)
- 客体的安全级 (f_2, f_4)

数据库管理系统的强制访问控制

强制访问控制——BLP模型的系统状态

$V = P_{(S \times O \times A)} \times \mu \times F$ 是状态的集合，状态 $v = (b, M, F)$ 用有序三元组表示，其中：

- b 是 $S \times O \times A$ 的子集，是当前访问集。
- $M \in \mu$ 是访问控制矩阵，它的第 i 行，第 j 列的元素 M_{ij} 是集合 A 的子集，表示在当前状态下，主体 S_i 对客体 O_j 所拥有的访问权限。
- $F = (f_1, f_2, f_3, f_4)$ 。其中， $f_1(s)$ 和 $f_3(s)$ 分别表示主体 s 的密级和范围集， $f_2(o)$ 和 $f_4(o)$ 分别表示客体 o 的密级和范围集。

数据库管理系统的强制访问控制

强制访问控制——BLP模型的安全特性

(1) 简单安全性——“向下读”

状态 $v=(b, M, F)$ 满足简单安全性，当且仅当对所有的 $(s, o, x) \in b$ ，有：

(1) $x=e$ 或 $x=a$ 或 $x=c$

或 (2) $(x=r$ 或 $x=w)$ 且 $(f_1(s) \geq f_2(o), f_3(s)$ 包含 $f_4(o))$ 。

主体 s 允许访问客体 o ，当且仅当主体的安全级高于等于客体的安全级。

BLP模型的安全特性定义了系统状态的安全性，体现了BLP模型的安全策略。

数据库管理系统的强制访问控制

强制访问控制——BLP模型的安全特性

(2) *-特性

状态 $v=(b, M, F)$ 满足*-特性，当且仅当：

对所有的 $s \in S$ ，若 $o_1 \in b(s: w, a)$ ， $o_2 \in b(s: r, w)$ ，则 $f_2(o_1) \geq f_2(o_2)$ ， $f_4(o_1)$ 包含 $f_4(o_2)$ ，其中：

符号 $b(s: x_1, \dots, x_n)$ 表示 b 中主体 s 对其具有访问特权 x_i ($1 \leq i \leq n$) 的所有客体的集合。

$$a) \quad o \in b(s: a) \Rightarrow f_2(o) \geq f_1(s);$$

$$b) \quad o \in b(s: w) \Rightarrow f_2(o) = f_1(s);$$

$$c) \quad o \in b(s: r) \Rightarrow f_1(s) \geq f_2(o);$$

此限制表明没有从高安全级主体到低安全级主体的直接信息流动

数据库管理系统的强制访问控制

强制访问控制——BLP模型的安全特性

(3) 自主安全性

状态 $v=(b, M, F)$ 满足自主安全性, 当且仅当对所有的 $(s_i, o_j, x) \in b$, 有 $x \in M_{ij}$ 。

系统安全的定义:

一个状态 v 如果满足上述三条性质, 那么 v 才是安全状态。

数据库管理系统的强制访问控制

强制访问控制——BLP模型的请求

请求集:

$R = S^+ \times RA \times S^+ \times O \times X$ 请求集, R 的元素是一个完整的请求其中 $S^+ = S \cup \Phi$, $X = A \cup \Phi \cup F$ 。

请求元素: $\langle \sigma_1, \gamma, \sigma_2, o_j, x \rangle$

设 $T = \{1, 2, \dots, t, \dots\}$ 离散时刻的集合 (标识)。用作请求序列, 结果序列和状态序列的下标;

请求序列: $X_1 = R^T = \{x | x: T \rightarrow R\}$, 其中元素 x 可表示为 $x = x_1 x_2 x_3 \dots x_t \dots$ 是一个请求序列。

每时刻有一请求, 构成请求序列, X_1 是请求序列集合;

数据库管理系统的强制访问控制

强制访问控制——BLP模型的结果序列

请求集:

$R = S^+ \times RA \times S^+ \times O \times X$ 请求集, R 的元素是一个完整的请求其中 $S^+ = S \cup \Phi$, $X = A \cup \{\Phi\} \cup F$ 。

请求元素: $\langle \sigma_1, \gamma, \sigma_2, o_j, x \rangle$

结果序列: $Y = D^T = \{y | y: T \rightarrow D\}$, 其中元素 $y = y_1 y_2 y_3 \dots y_t \dots$ 是一个结果序列, 每一时刻的请求导致一个判断 (或结果), 构成一个结果序列。 Y 是结果序列的集合;

状态序列: $Z = V^T = \{z | z: T \rightarrow V\}$, 其中元素 $z = z_1 z_2 z_3 \dots z_t \dots$ 是一个状态序列, 每一 $z_t \in V$, 表示时刻 t 时系统的状态。 Z 是状态序列的集合

数据库管理系统的强制访问控制

强制访问控制——BLP模型的状态转换规则

状态转换规则：系统状态转换由一组规则定义，一个规则 ρ 定义为：

$$R \times V \rightarrow D \times V$$

其中： R 是请求集， D 为判断集， V 是状态集。

解释：对于给定的一个状态和一个请求，系统产生一个判断和下一个状态，只有当 D 的取值为“yes”时，请求才被执行，状态才发生转换。

BLP模型的基本规则：

规则1～规则4：分别用于主体请求对客体的读(ϕ, g, si, oj, r)、添加(ϕ, g, si, oj, a)、执行(ϕ, g, si, oj, e)、和写(ϕ, g, si, oj, w)的访问权。

数据库管理系统的强制访问控制

强制访问控制——BLP模型的状态转换规则

BLP模型的基本规则概要:

规则1~规则4: 分别是主体请求对客体的读(ϕ, g, si, oj, r), 添加(ϕ, g, si, oj, a), 执行(ϕ, g, si, oj, e), 和写(ϕ, g, si, oj, w) 访问权。

规则5: 用于主体释放它对某客体的访问权(ϕ, r, si, oj, x)

规则6和规则7: 用于一个主体授予($s_\lambda, g, si, oj, r/w/a/e$)和撤销($s_\lambda, r, si, oj, r/w/a/e$)另一个主体对某客体的访问权。

规则8: 改变静止客体的密级和类别集。(ϕ, c, ϕ, oj, f^*)

规则9和规则10: 分别用于创建和删除(使之成为静止)一个客体。(ϕ, c, sj, oj, e) (ϕ, d, si, oj, ϕ) (ϕ, c, si, oj, ϕ)

数据库管理系统的强制访问控制

强制访问控制——BLP模型的状态转换规则

规则1: 主体 s_i 请求得到对客体 o_j 的 r 访问权

get-read $\rho_1(R_k, v) \equiv$

如果 $\sigma_1 \neq \varnothing$ or $\gamma \neq g$ or $x \neq r$ or $\sigma_2 = \varnothing$ then

$\rho_1(R_K, v) = (?, v)$

如果 $r \notin M_{ij}$ or $(f_1(s_i) < f_2(o_j) \text{ or } f_3(s_i) \not\sqsubseteq f_4(o_j))$

then $\rho_1(R_K, v) = (\text{no}, v)$

如果 $U_{\rho_1} = \{o | o \in b(s_i:w, a) \text{ and}$

$[f_2(o_j) > f_2(o) \text{ or } f_4(o_j) \not\sqsubseteq f_4(o)]\} = \varnothing$

then $\rho_1(R_K, v) = (\text{yes}, v^* = (b \cup \{(s_i, o_j, r)\}, M, f))$

else $\rho_1(R_K, v) = (\text{no}, v)$

end

数据库管理系统的强制访问控制

规则1 对主体 s_i 的请求作了如下检查:

1. 主体的请求是否适合于规则1;
2. O_j 的拥有者 (或控制者) 是否授予了 s_i 对 o_j 的读访问权;
- 3*. s_i 的安全级是否支配 o_j 的安全级;
- 4*. 在访问集 b 中, 若 s_i 对另一客体 o 有 w 访问权或 a 访问权, 是否一定有 o 的安全级支配 o_j 的安全级

数据库管理系统的强制访问控制

强制访问控制——BLP模型的状态转换规则

规则2: 主体 s_i 请求得到对客体 o_j 的 a 访问权

get-append: $\rho_2(R_K, v) \equiv$

如果 $\sigma 1 \neq \varnothing$ or $\gamma \neq g$ or $x \neq a$ or $\sigma 2 = \varnothing$,

则: $\rho_2(R_K, v) = (?, v)$

如果 $a \notin M_{ij}$, 则 $\rho_2(R_K, v) = (no, v)$

如果 $U_{\rho 2} = \{o | o \in b(s_i:r, w) \text{ and}$

$[f_2(o_j) < f_2(o) \text{ or } f_4(o_j) \not\subseteq f_4(o)]\} = \varnothing$

则 $\rho_2(R_K, v) = (yes, v^* = (b \cup \{(s_i, o_j, a)\}, M, f))$

否则 $\rho_2(R_K, v) = (no, v)$

end

数据库管理系统的强制访问控制

规则2对主体 s_i 的请求作了如下检查:

1. 主体的请求是否适合于规则2;
2. o_j 的拥有者（或控制者）是否授予了 s_i 对 o_j 的append访问权;
- 3*. 在访问集 b 中, 若 s_i 对另一客体 o 有 w 访问权或 r 访问权, 是否一定有 o 的安全级被 o_j 的安全级支配

数据库管理系统的强制访问控制

强制访问控制——BLP模型的状态转换规则

规则3: 主体 s_i 请求得到对客体 o_j 的 e 访问权

get-execute: $\rho_3(R_K, v) \equiv$

if $\sigma_1 \neq \phi$ or $\gamma \neq g$ or $x \neq e$ or $\sigma_2 = \phi$

then $\rho_3(R_K, v) = (?, v)$

if $e \notin M_{ij}$ then $\rho_3(R_K, v) = (\text{no}, v)$

else $\rho_3(R_K, v) = (\text{yes}, v^* = (b \cup \{(s_i, o_j, e)\}, M, f))$

end

***不必进行安全级的比较**

数据库管理系统的强制访问控制

强制访问控制——BLP模型的状态转换规则

规则4: 主体 s_i 请求得到对客体 o_j 的 w 访问权

get-write: $\rho_4(R_K, v) \equiv$

if $\sigma_1 \neq \varnothing$ or $\gamma \neq g$ or $x \neq w$ or $\sigma_2 = \varnothing$

then $\rho_4(R_K, v) = (?, v)$

if $w \in M_{ij}$ or $[f_1(s_i) < f_2(o_j) \text{ or } f_3(s_i) \sqsubseteq f_4(o_j)]$

then $\rho_4(R_K, v) = (\text{no}, v)$

if $U_{\rho_4} = \{o \mid o \in b(s_i:r) \text{ and } [f_2(o_j) < f_2(o) \text{ or } f_4(o_j) \sqsupset f_4(o)]\}$

$\cup \{o \mid o \in b(s_i:a) \text{ and } [f_2(o_j) > f_2(o) \text{ or } f_4(o_j) \sqsubset f_4(o)]\}$

$\cup \{o \mid o \in b(s_i:w) \text{ and } [f_2(o_j) \neq f_2(o) \text{ or } f_4(o_j) \neq f_4(o)]\}$

$= \varnothing$

then $\rho_4(R_K, v) = (\text{yes}, v^* = (b \cup \{(s_i, o_j, w)\}, M, f))$

else $\rho_4(R_K, v) = (\text{no}, v)$ end

数据库管理系统的强制访问控制

安全性检查类似于规则1:

1. s_i 的请求是否适用于规则4;
2. s_i 是否被自主地授予了对 o_j 的 w 权;
3. s_i 的安全级是否支配 o_j 的安全级;
4. *-特性的检查:
 - .在 b 中, 若 s_i 已对某一客体 o 有读权, 则 o_j 的安全级必须支配 o 的安全级
 - .在 b 中, 若 s_i 已对某一客体 o 有添加权, 则 o_j 的安全级必须受 o 的安全级支配
 - .在 b 中, 若 s_i 已对某一客体 o 有写权, 则 o_j 的安全级必须等于 o 的安全级

数据库管理系统的强制访问控制

强制访问控制——BLP模型的状态转换规则

规则5: 主体 s_i 请求释放对客体 o_j 的r或w或e或a访问权
release-read/write/append/execute:

$\rho_5(R_K, v) \equiv$

if $(\sigma_1 \neq \varnothing) \text{ or } (\gamma \neq r) \text{ or } (x \neq r, w, a \text{ and } e) \text{ or } (\sigma_2 = \varnothing)$

then $\rho_5(R_K, v) = (?, v)$

else $\rho_5(R_K, v) = (\text{yes}, v^* = (b - \{(s_i, o_j, x)\}, M, f))$

end

*访问权的释放不会对系统安全造成威胁，因此不需要做安全检查

数据库管理系统的强制访问控制

强制访问控制——BLP模型的状态转换规则

规则6: 主体 s_λ 请求授予主体 s_i 对客体 o_j 的r或w或e或a访问权

give-read/write/append/execute

$\rho_6(R_K, v) \equiv$

if $(\sigma_1 \neq s_\lambda \in S) \text{ or } (\gamma \neq g) \text{ or } (x \neq r, w, a \text{ and } e) \text{ or } (\sigma_2 = si)$

then $\rho_6(R_K, v) = (?, v)$

if $x \notin M_{\lambda j} \text{ or } c \notin M_{\lambda j}$ then $\rho_6(R_K, v) = (no, v)$

else $\rho_6(R_K, v) = (yes, (b, M^{\oplus}[x]_{ij}, f))$

end

s_λ 的请求只涉及自主访问控制中的授权，规则6除了检查请求是否适用于规则6外，只做自主安全性有关的检查： s_λ 自身必须同时具有对客体 o_j 的x权和控制权c

数据库管理系统的强制访问控制

强制访问控制——BLP模型的状态转换规则

规则7： 主体 s_λ 请求撤销主体 s_i 对客体 o_j 的r或w或e或a访问权
rescind-read/write/append/execute:

$\rho7(R_K, v) \equiv$
if $(\sigma1 \neq s_\lambda \in S)$ or $(\gamma \neq r)$ or $(x \neq r, w, a \text{ and } e)$ or $(\sigma2 = si)$
then $\rho7(R_K, v) = (?, v)$
if $x \notin M_{\lambda j}$ or $c \notin M_{\lambda j}$ then $\rho7(R_K, v) = (no, v)$
else $\rho7(R_K, v) = (yes, (b - \{(si, oj, x)\}, M \ominus [x]_{ij}, f))$
end

规则7只涉及自主安全性，系统要求 s_λ 必须对 o_j 同时具有x权和c权，才能对 s_i 的x权予以撤销

数据库管理系统的强制访问控制

强制访问控制——BLP模型的状态转换规则

规则8： 改变静止客体的安全级

change-f: $R_K = (\varphi, c, \varphi, oj, f^*)$ (静止客体新建后安全级由系统确定)

$\rho_8(R_K, v) \equiv$

if $(\sigma_1 = \varphi) \text{ or } (\gamma \neq c) \text{ or } (\sigma_2 = \varphi) \text{ or } f^* \notin F$

then $\rho_8(R_K, v) = (?, v)$

if $f^*_1 \neq f_1 \text{ or } f^*_3 \neq f_3 \text{ or } [f^*_2(oj) \neq f_2(oj) \text{ or}$

$f^*_4(oj) \neq f_4(oj) \text{ for some } j \in A(m)]$

then $\rho_8(R_K, v) = (\text{no}, v)$ ——不能改变活动主客体安全级

else $\rho_8(R_K, v) = (\text{yes}, (b, M, f^*))$

end 注: $A(m)$ 表示活动客体的集合

静止客体是指被删除了的客体，其客体名可以被系统的主体重新使用，重用时，客体安全级为创建者的安全级

数据库管理系统的强制访问控制

规则8的安全性要求:

新定义的安全级 $f^*=(f_1^*, f_2^*, f_3^*, f_4^*)$ 不能改变系统中主体的安全级, 也不能改变活动客体的安全级, 此时新状态 v^* 用 f^* 代替原状态 v 中的 f

数据库管理系统的强制访问控制

强制访问控制——BLP模型的状态转换规则

规则9: 主体 s 请求创建客体 o_j

create-object:

$\rho_9(R_K, v) \equiv$

if $\sigma 1 = \varphi$ or $\gamma \neq c$ or $\sigma 2 = si$ or $(x \neq e \text{ and } \varphi)$

then $\rho_9(R_K, v) = (?, v)$

if $j \in A(m)$ then $\rho_9(R_K, v) = (no, v)$

if $x = \varphi$ then $\rho_9(R_K, v) = (yes, (b, M^{\oplus}[\{r, w, a, c\}]_{ij}, f))$

——创建不可执行客体

else $\rho_9(R_K, v) = (yes, (b, M^{\oplus}[\{r, w, a, c, e\}]_{ij}, f))$

end

要求 s_i 所创建的客体不能是活动着的客体

数据库管理系统的强制访问控制

强制访问控制——BLP模型的状态转换规则

规则10： 主体 s 请求删除客体 o_j

delete-object:

$\rho_{10}(R_K, v) \equiv$

if $\sigma_1 = \varnothing$ or $\gamma \neq d$ or $\sigma_2 = \varnothing$ or $x \neq \varnothing$

then $\rho_{10}(R_K, v) = (?, v)$

if $c \notin M_{ij}$ then $\rho_{10}(R_K, v) = (no, v)$

else $\rho_{10}(R_K, v) = (yes, (b, M \ominus \{r, w, a, c, e\}_{ij, 1 \leq i \leq n}, f))$

end

- s_i 必须对 o_j 具有控制权才能删除 o_j （这里拥有权和控制权一致）；
- o_j 被删除后， o_j 成为静止客体，此时访问矩阵的第 j 列，即 o_j 所对应的列中各元素必须全部清空，不包含任何权限

数据库管理系统的强制访问控制

强制访问控制——BLP模型的系统的定义

$$\mathbf{R} \times \mathbf{D} \times \mathbf{V} \times \mathbf{V} = \{(\mathbf{r}_K, \mathbf{d}_m, \mathbf{v}^*, \mathbf{v}) \mid \mathbf{r}_K \in \mathbf{R}, \mathbf{d}_m \in \mathbf{D}, \mathbf{v}^*, \mathbf{v} \in \mathbf{V}\}$$

即，任意一个请求，任意一个结果（判断）和任意两个状态都可组成一个上述的有序四元组，这些有序四元组便构成集合 $\mathbf{R} \times \mathbf{D} \times \mathbf{V} \times \mathbf{V}$ 。

数据库管理系统的强制访问控制

强制访问控制——BLP模型的系统的定义

设 $\omega = \{\rho_1, \rho_2, \dots, \rho_s\}$ 是一组规则的集合，定义 $W(\omega)$ 被包含于 $R \times D \times V \times V$ 如下：

- (1) $(r_k, ?, v, v) \in W(\omega)$ iff 对每个 $i, 1 \leq i \leq s, \rho_i(r_k, v) = (?, v)$
- (2) $(r_k, \text{error}, v, v) \in W(\omega)$ iff 存在 $i_1, i_2, 1 \leq i_1, i_2 \leq s$, 使得对于任意的 $v^* \in V$ 有 $\rho_{i_1}(r_k, v) \neq (?, v^*)$ 且 $\rho_{i_2}(r_k, v) \neq (?, v^*)$ 。

——有多条规则适合

- (3) $(r_k, d_m, v^*, v) \in W(\omega), d_m \neq ?, d_m \neq \text{error}$, iff 存在唯一的 $i, 1 \leq i \leq s$, 使得对某个 v^* 和任意的 $v^{**} \in v, \rho_i(r_k, v) \neq (?, v^{**})$,

$\rho_i(r_k, v) = (d_m, v^*)$

——唯一规则适合

数据库管理系统的强制访问控制

$(r_k, d_m, v^*, v) \in W(\omega)$, 则该四元组满足上述定义中 (3条) 的某一条, 亦即意味着在状态 v 下, 发出某请求 r_k 后, 按照某条规则, 其结果为 d_m , 状态 v 转换成状态 v^* 或者保持状态不变

数据库管理系统的强制访问控制

强制访问控制——BLP模型的系统表示

系统表示为 $\Sigma(R, D, W(\omega), z_0)$ ，定义为：

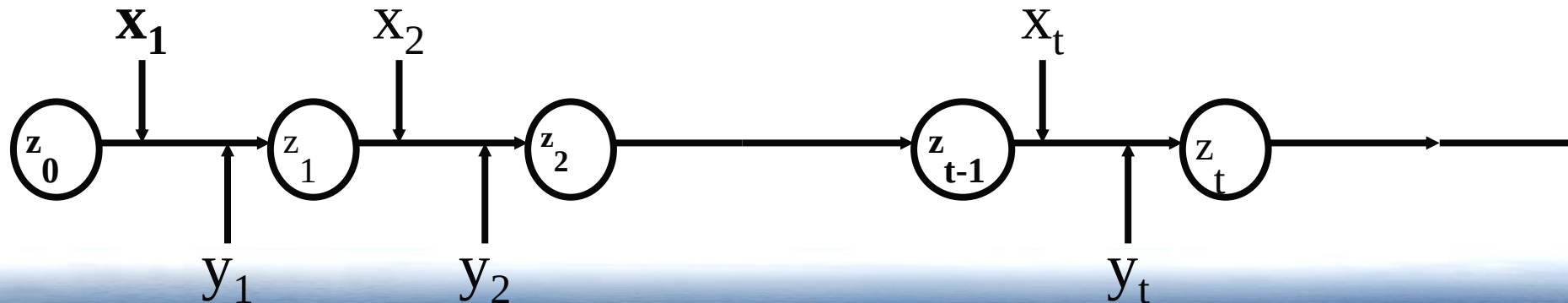
$\Sigma(R, D, W(\omega), z_0)$ 被包含于 $X \times Y \times Z$ ，只含有其中一部分有序三元组， $X \times Y \times Z$ 中的有序三元组

$(x, y, z) \in \Sigma(R, D, W(\omega), z_0)$ ，iff 对每一个 $t \in T$ ，

$(x_t, y_t, z_t, z_{t-1}) \in W(\omega)$ 。

z_0 是系统的初始状态，通常表示为 (ϕ, M, f)

$x = x_1 x_2 \dots x_t \dots$, $y = y_1 y_2 \dots y_t \dots$ $z = z_1 z_2 \dots z_t \dots$



数据库管理系统的强制访问控制

强制访问控制——BLP模型的系统安全定义

1. 安全状态

一个状态 $v=(b, M, f) \in V$ ，若它满足自主安全性，简单安全性和*—特性，那么这个状态就是安全的。

2. 安全状态序列

设 $z \in Z$ 是一状态序列，若对于每一个 $t \in T$ ， $z \in t$ 都是安全状态，则 z 是安全状态序列。

数据库管理系统的强制访问控制

强制访问控制——BLP模型的系统安全定义

3. 系统的一次安全出现

$(x, y, z) \in \Sigma(R, D, W(\omega), z_0)$ 称为系统的一次出现。

若 (x, y, z) 是系统的一次出现，且 z 是一安全状态序列，则称 (x, y, z) 是系统 $\Sigma(R, D, W(\omega), z_0)$ 的一次安全出现。

4. 安全系统

若系统 $\Sigma(R, D, W(\omega), z_0)$ 的每次出现都是安全的，则称该系统是一安全系统。

数据库管理系统的强制访问控制

强制访问控制——BLP模型的有关安全的结论

1. 这十条规则都是安全性保持的。（即若 v 是安全状态，则经过这十条规则转换后的状态 v^* 也一定是安全状态）
2. 若 z_0 是安全状态， ω 是一组安全性保持的规则，则系统 $\Sigma(R, D, W(\omega), z_0)$ 是安全的。

说明BLP模型所描述的系统是一个安全的系统。

数据库管理系统的强制访问控制

思考题

1. BLP模型是一个通用的模型，可以用于任何系统，如何在数据库中实现这个模型？

提示：可以有两种实现方式

- 在DBMS内核实现；
- 在DBMS外层实现，依赖于操作系统实施强制访问控制。

数据库管理系统的强制访问控制

DBMS的强制访问控制设计——DBMS实施MAC要实现强制访问控制要解决哪些问题？

1. 运行环境
 2. 系统安全级的设置
 3. 主体的标记
 4. 客体的标记
 5. 强制访问控制的检查及其与自主访问控制之间的关系
- 
- 谁来实施？

数据库管理系统的强制访问控制

DBMS的强制访问控制设计——DBMS实施MAC

1. 多级安全DBMS的运行环境要求

- 运行在多级安全操作系统上（主体：进程，客体：文件、目录）
- 多级安全DBMS的安全机制与操作系统的多级安全机制一起组合成**统一的安全机制**
- 此时DBMS以多级安全操作系统中的**进程**运行，按照DBMS实现机制的不同，DBMS运行方式会有所差别——**DBMS进程是OS上的可信进程**（可以访问多个安全级的数据文件）
- 用户的应用程序也是以操作系统中的**进程**运行——**每个用户进程会有个安全级**

数据库管理系统的强制访问控制

DBMS的强制访问控制设计——DBMS实施MAC

2. 系统安全级的设置

设置**DBMS安全级**的时机：两种方式

- 初始化数据库时指定（安装时）——数据库文件在OS中以哪个安全级存放？——最高安全级
- 启动系统后，使用设置安全级的语句指定（扩展SQL实现）
- 与操作系统安全级之间的关系：
 - 受**操作系统安全级**的限制
 - 安全级的名可以和操作系统的安全级不同，但是要建立和操作系统安全级之间的对应关系

系统安全级的存放：存放在**数据字典**中

DBMS进程的安全级：在OS可信进程安全级的范围内

存放系统安全级的数据字典：**SYSLEVELS, SYSCATEGORY**

数据库管理系统的强制访问控制

DBMS的强制访问控制设计——DBMS实施MAC

3.主体的标记

DBMS中的主体——可以按用户去理解，表现形式为会话，在DBMS中是任务，任务实际上代表的是用户的行为

- 代表用户行为的任务，通过操作系统（OS）进程产生，在用户登录系统时初始化为客户端OS进程的代表，表现形式是代表用户的会话。

用户帐户许可：代表了用户账户的许可信息，是DBMS管理下某个指定用户可访问信息的最高安全级；（DBMS保存在字典表中，和帐户信息保存在一起）

与OS用户许可的关系： 分别维护，不必一致；但有映射关系

数据库管理系统的强制访问控制

DBMS的强制访问控制设计——DBMS实施MAC

3.主体的标记

- 主体标记初始化成与当前执行的OS进程的标记值一致——**主体的当前安全级**
- 一般用户进程只在一个级别上建立标记（可信进程除外），且这个级别必须被系统表中存储的用户安全许可所支配

安全许可的存放：和用户帐户信息一起存放在**数据字典**

谁来实施主体的标记？（由DBA来完成是否可以？）此时数据字典的安全级怎样设置？（为便于管理设置为**系统最低**）

数据库管理系统的强制访问控制

DBMS的强制访问控制设计——DBMS实施MAC

4. 客体的标记

DBMS中的客体：包含着能在不同主体间共享信息的实体，一般来说，客体都有属主。

- DBMS中客体标记粒度的选择

- .. 库（多库的概念下）

- ..表

- ..属性

- ..元组

- ..元素

- 粒度越细，系统开销越大，现有的**商用DBMS最小标记粒度到元组级**

数据库管理系统的强制访问控制

DBMS的强制访问控制设计——DBMS实施MAC

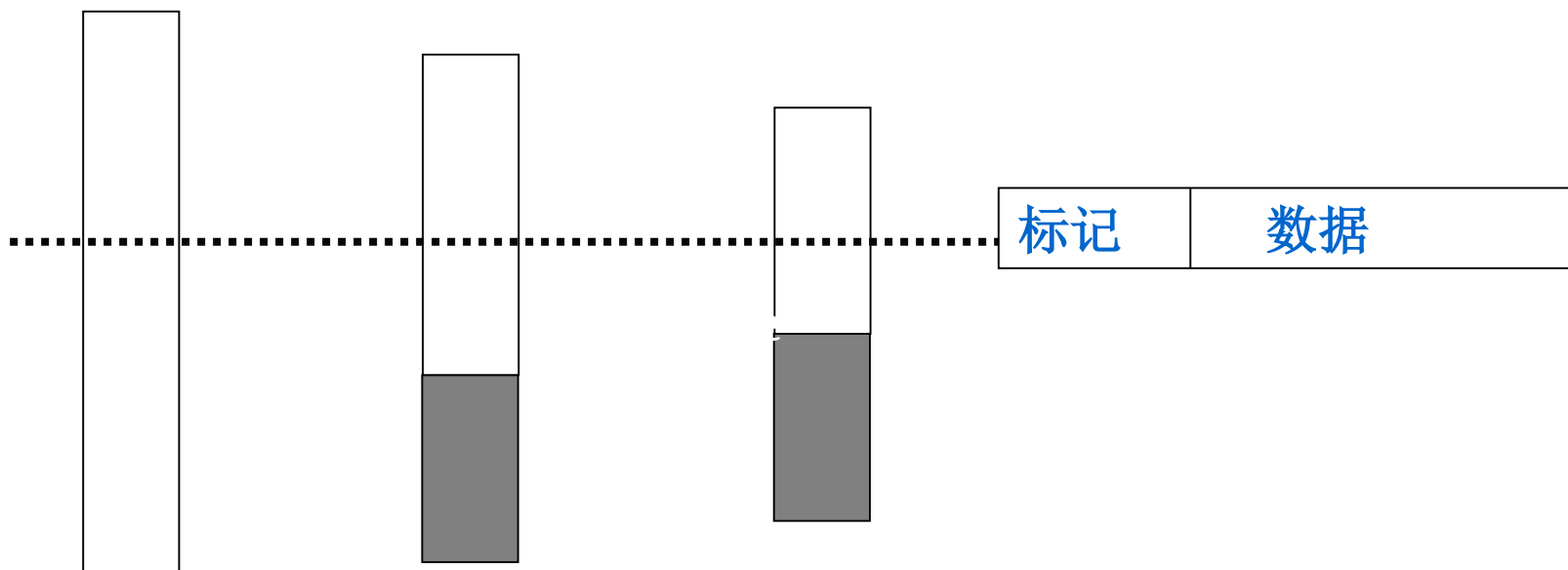
4.客体的标记——表级粒度

- 每个表都有一个安全级：由表的创建者指定（保存在表的字典中）
- 每个表的安全级支配数据库的安全级（现代数据库系统普遍支持多库概念）
- RVM实现简单，只要遵循BLP模型的“向下读”、“向上写”规则即可
- 表的标记的管理和维护问题？（谁来设置和修改？）
- 与自主访问控制之间的关系：都要起作用，检查有先后
- 表级粒度的系统，使用时存在哪些问题？——表内数据安全级相同

数据库管理系统的强制访问控制

DBMS的强制访问控制设计——DBMS实施MAC

4. 客体的标记——元组级粒度（容器的概念）



数据库管理系统的强制访问控制

DBMS的强制访问控制设计——DBMS实施MAC

4. 客体的标记——元组级粒度（容器的概念）

元组级标记的规则 ——增加存放安全级的列

- 数据库的初始安全级等于创建它的用户的安全级
- 表的初始安全级 = 创建者的当前安全级（称为表的最低安全级），并且支配数据库的安全级，受系统最高安全级的支配
- 表中数据的安全级：大于或等于表的初始安全级，等于插入行的用户的安全级
- 访问表的主体的安全级支配表的最低安全级
- 一般情况下，元组的安全级不可以修改，但是表的最低安全级可以修改——出于系统维护的需要，由系统安全员修改

数据库管理系统的强制访问控制

DBMS的强制访问控制设计——DBMS实施MAC

5. 强制访问控制权限检查——遵循BLP模型的检查

- 仅当主体的安全级**支配** ($\geq$) 客体的安全级标记时，允许**读**访问；
 - 仅当主体的安全级**等于**客体的安全级时，允许**写**访问；
- 为什么要这样改？

6. DBMS自主访问控制与强制访问控制之间的关系

- 两者同时发生作用
- 哪一个先检查？自主访问控制先检查

***系统先进行DAC检查，对通过DAC检查的允许存取的数据库对象再由系统自动进行MAC检查，只有通过MAC检查的数据库对象方可存取。**

数据库管理系统的强制访问控制

DBMS的强制访问控制设计——DBMS实施MAC

7. DBMS强制访问控制需要解决的其他问题

- 特权问题：出于系统维护的需要，一般DBMS往往会设置一些特权(向上读、向下写)
- 存储客体的管理：数据库、表、行的管理
- 会产生行级多实例：DBMS对行级多实例的处理方式不同
- 数据迁移问题（涉及数据字典、标记的导入、导出问题）

行级多实例：具有主关键字唯一性约束的表中，允许出现主关键字相同而安全级不同的元组

讨论的问题：视图的处理方法？——思路

数据库管理系统的强制访问控制

BLP模型的限制

问题：BLP模型中，当数据的标记中，**范围越大**，能够访问它的用户越少。——安全性好

例：工资数据标记为（机密，{财务处，人事处}），财务处的用户不能访问它，人事处的用户也不能访问它，只有主管财务处和人事处两个部门的领导才能访问它，限制了访问的用户。

实际应用的要求：财务处和人事处的用户都能访问标记为（机密，{财务处，人事处}）的数据

推理通道和隐通道

信息流通道 —— 推理通道和隐通道

推理通道：用户使用能够查看到的库内的数据，根据自己的知识，推导数据库内的无权查看的机密数据；

一个用户即可完成，并且往往和数据库结构设计、DBMS没有记录访问的历史数据有关。

隐通道：需要两个主体分别在低安全级和高安全级上共同合作，还有一种编码模式和同步方式。

目前基于强制访问控制的引用监控机制不能防止推理和隐通道的发生

编码模式：两个用户之间协商好的数据传送方式。

同步方式：传递完一位以后，怎样传递下一位的方法。

需要两个用户协作完成。

推理通道和隐通道

信息流通道——隐通道的基本概念

隐通道：是系统的一个主体用违反系统安全策略的方式传送信息给另一主体的机制。在一个系统（操作系统或数据库管理系统）中，给定一个安全策略模型 M 及其解释 $I(M)$ ， $I(M)$ 中的任何两个主体 $I(S_h)$ 和 $I(S_i)$ 间的潜在通信是隐蔽的当且仅当 M 的相应主体 S_h 和 S_i 间的通信在 M 中是非法的。

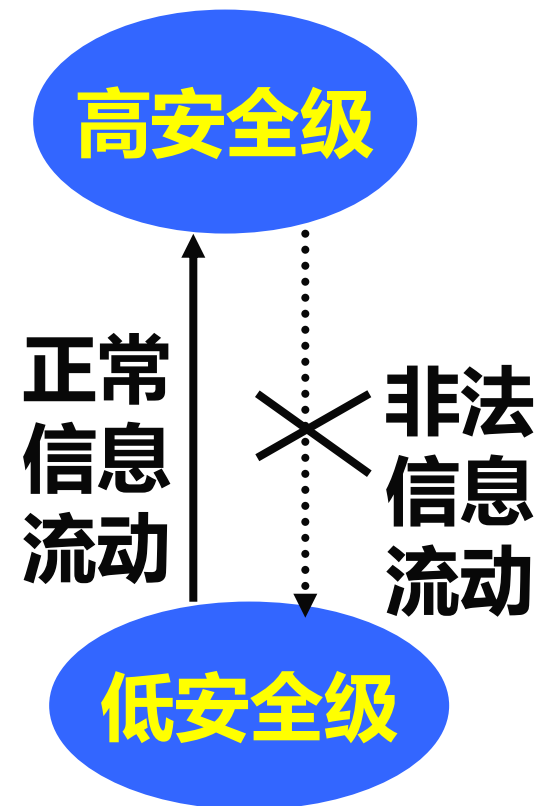
推理通道和隐通道

信息流通道 —— 隐通道的基本概念

采用BLP模型的系统中，根据安全策略，正常和非法的信息流动

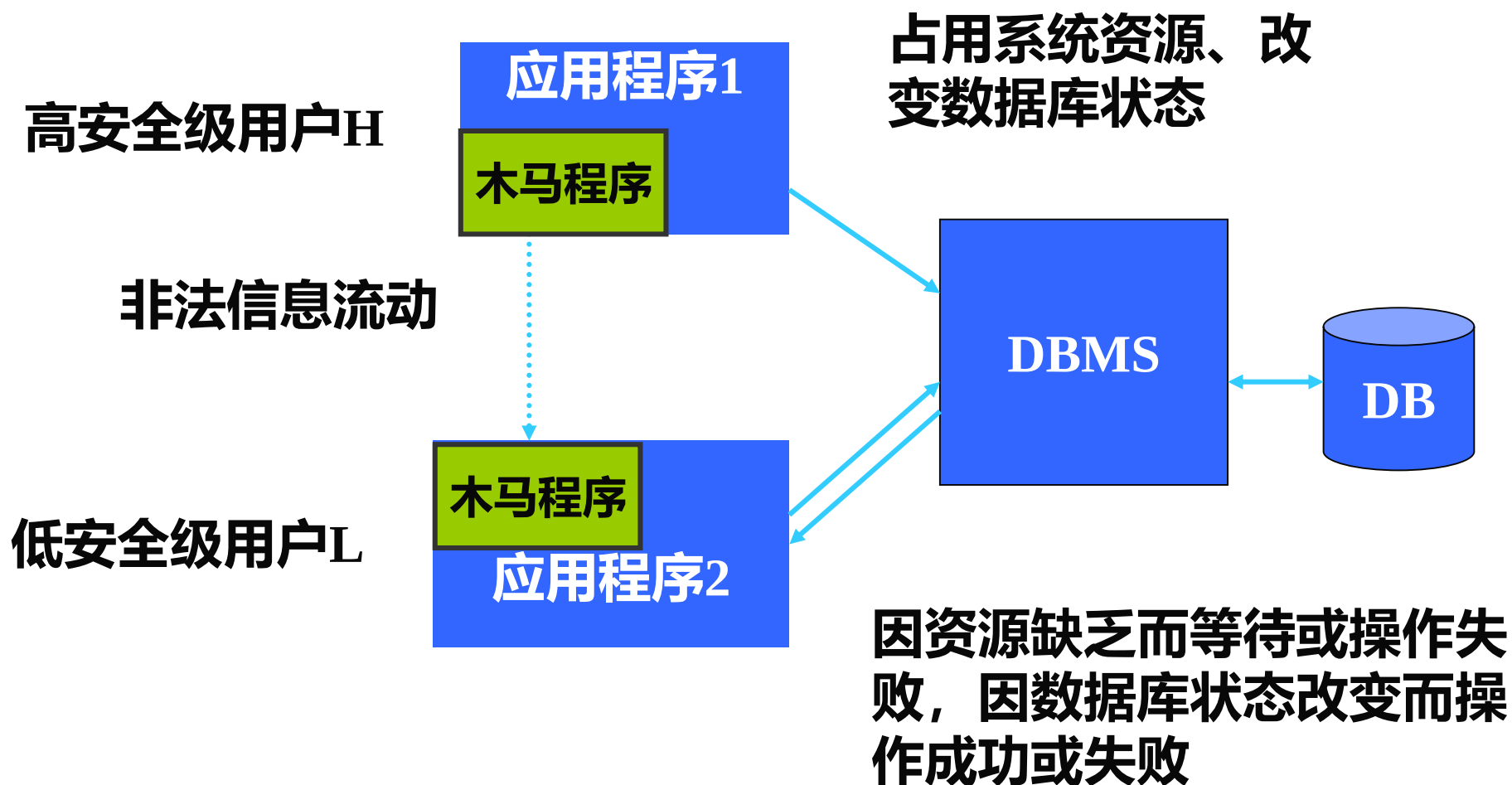
仅当主体的标记支配客体的标记时，才允许读访问；

仅当主体的标记等于客体的标记时，才允许写访问。



推理通道和隐通道

信息流通道 —— 隐通道的基本概念



数据库管理系统中的隐通道

完整性约束引起的隐通道

① 主关键字约束

在行级粒度标记的系统中：

```
CREATE TABLE TEST1(COL1 INT PRIMARY KEY,  
                      COL2 CHAR(9));
```

插入语句：**INSERT INTO TEST1 VALUES(999, 1);**

（1）高安全级主体插入关键字为999的元组 ——成功

低安全级主体插入关键字为999的元组——失败

（2）高安全级主体删掉关键字为999的元组

低安全级主体插入关键字为999的元组——成功

编码模式：失败编码为0，成功编码为1

数据库管理系统中的隐通道

完整性约束引起的隐通道

② 唯一性索引约束

```
create table t02(col1 int not null,  
                col2 char(9), col3 char(9) not null);
```

```
create unique index s2 on t02 (col3, col1);
```

插入语句: **INSERT INTO T02 VALUES(100, ?, 'string');**

形成隐蔽通道的发送者操作系列:

1 (INSERT INTO T02 VALUES(100, ?, 'string');).

形成隐蔽通道的接收者操作系列:

2 (INSERT INTO T02 VALUES(100, ?, 'string');)

编码模式: 失败编码为0, 成功编码为1

数据库管理系统中的隐通道

数据字典引起的隐通道

系统表中包含有关于系统内数据的描述信息。利用这些信息可以形成隐通道。

除了上述权限信息外，还包括视图、表上的引用约束引起的隐通道（客体存在通道）、基于表上视图个数引起的隐通道

Ex: 授权信息

高安全级主体：授权低安全级主体查询高安全级主体创建的表

低安全级主体：按照BLP模型，不能查询该表，但是从其对数据字典的查询结果中可以看到该表的存在。

编码模式：查看到有表的权限——编码为1

查看不到有表的权限——编码为0

数据库管理系统中的隐通道

全局变量的资源耗尽型的隐蔽通道

数据库系统中有些资源是有限的，当高安全级主体全部占用这些资源时，也会因低安全级主体申请不到这些资源而形成隐通道

问题：数据库系统两个以上用户操作，隐通道不存在？

数据库管理系统中的隐通道

并发控制的上锁机制引起的定时隐蔽通道

两个主体按照下列顺序操作：

1. 高安全级主体产生的事务：上锁读 x ， $r_2[x]$
2. 低安全级主体产生的事务：请求上写锁写 x ， $w[x]$

此时低安全级主体产生的事务将等待

3. 高安全级事务通过调整占用读锁的时间，发送信息给低安全级主体。

这个隐通道的解决涉及到事务可串行化问题。

时间戳算法也存在类似的问题。

隐通道的分类

(1) 存储隐通道：隐通道的发送者和接收者之间事先约定某种编码方式，当发送者利用系统正常操作，修改系统的**某个共享资源属性**而接收者能监测到这种修改时这个隐通道是存储隐通道

(2) 定时隐通道：当发送者通过**调整接收者的响应时间**来传送信息时这个隐通道是定时隐通道。

隐通道的分类

存储隐通道的判定准则

- 发送者和接收者进程必须都能够访问同一共享实体
- 发送者进程必须能使共享实体改变
- 接收者实体必须能检测共享实体的变化
- 必须有一种机制在发送者和接收者之间进行初始化通信和同步

隐通道的分类

定时隐通道的判定准则

- 发送者和接收者进程必须都能够访问同一共享实体
- 发送者进程必须都能访问一时间参考
- 发送者进程必须能够修改所花的时间，而接收者进程能检测到共享实体的变化
- 必须有一种机制在发送者和接收者之间进行初始化通信和同步

隐通道的分类

信息流通道——隐通道与信号通道之间的区别

- （1）信号通道是基本算法和协议中固有的非法信息流动，因此它会出现出现在任何具体的实现中。
- （2）隐通道则是某个特定实现引入的，不是一般算法或协议固有的。即使基本算法不含隐通道，在一个具体的实现中仍然可能出现隐通道。

隐通道的分类

信息流通道——隐数据库管理系统中的隐通道产生原因

- (1) 与TCB的缺陷有密切联系。隐通道是由系统在设计、实现安全模型时的漏洞引起的。

TCB（计算机内保护装置的总体）

- (2) 系统资源的共享。发送者必须直接或间接地修改某个存储位置属性，即某个系统资源属性的值；同时接收者必须能看到这个系统资源属性的改变，即发送者和接收者之间必须共享某个系统资源。

隐通道的分类

信息流通道——隐通道与强制访问控制之间的关系

隐通道只与强制访问控制策略有关，而与自主访问控制策略无关。——基本概念

在只有自主访问控制策略的系统中，不能防止特洛伊木马以合法的操作取得敏感信息，若再去考虑如何利用隐通道非法泄露信息就毫无意义了。

问题：1. SQL-SERVER、ORACLE中是否存在隐通道？为什么？
2. 隐通道 = “后门”（或者称天窗）吗？

隐通道的分类

作业题1: 限选7人

高安全级用户利用多级安全数据库管理系统的引用完整性引起的隐通道将自己的用户名、口令传递给低安全级用户。

要求设计两个用户之间: 编码方式、同步方式

建议: 使用达梦数据库 <http://www.dameng.com> 安全版

提示: 同步方式, 高安全级用户准备好自己的数据后, 可以利用另一个隐通道通知低安全级用户准备完毕

同步方式也使用引用完整性引起的隐通道

要求: 演示程序, 用ppt报告你的实现机制

数据库管理系统的基于角色的访问控制

基于角色的访问控制的基本思想

- 根据**应用的需要**创建相应的角色，并将用户分派到相应的角色集合中，根据应用需要给用户分配一些角色
- 权限与角色相映射，给角色分配一些权限
- 角色与用户联系起来，从而将角色与权限联系起来。而不是将访问权限直接分配给用户
- 可根据应用需要将分配给用户的某些角色撤消，也可以将分配给角色的某些权限撤消
- 根据系统中制定的安全策略进行权限-角色分配、角色-用户分配

数据库管理系统的基于角色的访问控制

基于角色的访问控制的目标——简化权限管理

RBAC支持三个著名的安全原则

最小特权原则

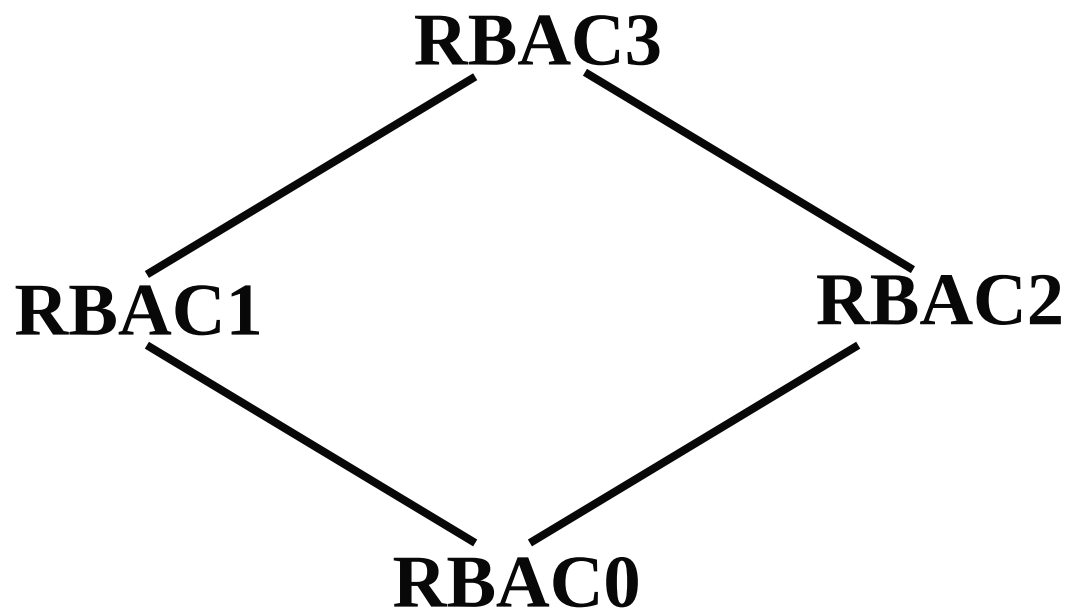
职责分离原则

数据抽象原则

RBAC的特点

- 与应用无关
- 在不同的系统配置下可以有不同的安全控制功能——可以构造自主访问控制类型、强制访问控制类型、自主和强制访问控制类型的系统

RBAC96模型简介



RBAC96模型家族系列关系图

RBAC96模型简介

基本模型RBAC0

基本模型包含四个基本要素：用户（U）、角色（R）、权限（P）和会话（S）。

用户与角色之间是多对多关系

角色与权限之间是多对多关系

RBAC模型中的授权就是将客体的访问权限在可靠的控制下，连带角色所需要的操作一起提供给角色所代表的用户

RBAC96模型简介

会话及其与用户角色之间的关系

- 用户进入系统得到自己控制时，就得到一个会话，它可以代表用户的行为
- 会话是动态产生的，从属于一个用户，一个用户可以打开多个会话
- 若定义过角色与用户的关系，会话将它所代表的用户映射到多个角色
- 会话激活的角色是该用户的全部角色的子集
- 会话内可以获得全部被激活的角色所代表的授权
- 会话的特点：便于实施最小特权原则

RBAC96模型简介

角色的层次结构RBAC1 (RH)

思想：在一般的单位或组织中，职权通常是具备偏序关系的，角色的层次关系体现了上级领导所得到的信息访问权限高于下级职员的权利

特点：

- RBAC1对层次角色的支持包括了对偏序模型的支持
- RH表示上一级角色可以继承下一级角色的访问权限

RBAC96模型简介

约束模型RBAC2

模型的特点：

- RBAC1和RBAC2之间是互不相干的，是不可比较的
- 约束即角色间的相互排斥性
- 约束是通过将一项敏感任务分给多个用户完成，或限制对系统重要信息的操作，以减轻单个用户出错对系统造成的影响
- 约束的例子：公司的会计和出纳，任何一个公司都绝不会允许同时分配给某一个具体人员这两个角色。

RBAC96模型简介

综合模型RBAC3

综合模型RBAC3集中了RBAC1和RBAC2的所有特点，适合于在非常复杂的应用背景下部署和实施基于角色的访问控制。

RBAC96模型简介

RBAC模型的SQL支持

将指定的特权权限授予/回收用户或者角色 权限→角色

(1) **GRANT** <特权> **ON** [<对象类型>] <数据库对象> **TO** <用户或角色>{,<用户或角色>} [**WITH GRANT OPTION**]

<对象类型>::= **TABLE | VIEW**

<用户或角色>::= <用户名> | <角色名>

(2) **REVOKE** [**GRANT OPTION FOR**] <特权> **ON** [<对象类型>] <数据库对象> **FROM** <用户或角色> {,<用户或角色>}

RBAC96模型简介

RBAC模型的SQL支持

将指定的角色授予/回收 用户或者角色 角色→角色，形成角色的层次关系

(1) **GRANT** <角色名>, { <角色名>} **TO** <用户或角色>{, <用户或角色>} [**WITH GRANT OPTION**]

不支持循环授权

(2) **REVOKE** <角色名>{,<角色名>} **FROM** <角色名或用户名>
>

数据库管理系统访问控制

细粒度的访问控制在DBMS层的实现

行级访问控制 —— 视图，为不同的用户建立不同的视图

```
CREATE VIEW John_View AS
```

```
SELECT s_id, c_id, grade FROM Grade
```

```
WHERE s_id = 200701
```


数据库管理系统访问控制

细粒度的访问控制在DBMS层的实现

行级访问控制 —— 视图

问题:

- 当系统中的用户数量很多并且变化很大的情况下对视图进行管理和维护非常复杂
- 应用程序操作的对象是视图而非原始的表，应用程序受到这些视图的制约，同样不易于修改和复用，并且有被绕过的可能

数据库管理系统访问控制

细粒度的访问控制在DBMS层的实现

行级访问控制 —— 动态视图
对同类用户定义带参数的视图

例: **create view MyGrades as select * from Grades where s_id=\$user-id.**

问题:

对静态视图的部分改善, 静态视图中存在的问题依然存在, 尤其是第二点

数据库管理系统访问控制

细粒度的访问控制在DBMS层的实现

行级访问控制 —— 动态修改SQL语句

根据FGAC策略，对用户提交的SQL语句进行动态修改

例: **select * from Grades;**

改写后: **select * from Grades
 where s_id=\$user_id;**

问题:

语义问题——可能得到的结果与用户期待的不同

效率问题——被改写后的SQL语句可能会很复杂

数据库管理系统访问控制

细粒度的访问控制（FGAC）在商用系统中的实现

一些商用DBMS支持FGAC

——Oracle(VPD), DB2, Sybase

- 方法一：查询重写
 - .. 根据访问控制的要求产生谓词
 - ..将谓词附加在用户的一个查询之后
 - ..查询引擎执行被修改后的查询
 - ..多个策略之间的关系：and 或者 or

存在问题：难以写出关于表的联接、引用等复杂情形下的谓词，某些情形下用户被授权查看某些数据，但是一个正确的查询由于增加了一个谓词而可能成为不合法的查询

数据库管理系统访问控制

目前的实现方法存在的共同的问题

语义问题

例如: **select avg(grade) from Grades;**

(1) 视图: **select avg(grade) from MyGrades;**

(2) 动态修改SQL:

**select avg(grade) from Grades where
s_id=\$user-id;**

该学生认为自己的平均成绩为所有学生的平均成绩。

数据库管理系统访问控制

目前的实现方法存在的共同的问题

效率问题

当视图定义很复杂，或者动态修改后需要添加的语句很复杂，若此时用户提交的SQL语句很复杂，这样的执行效率是很低的。而且这种情况下极有可能造成部分语句重复。

例如：假设学号为200701的学生执行：

```
select * from Grades where s_id=200701;
```

改写后：**select * from Grades where s_id=200701 and s_id=&user-id;**

Summary of The Class



- 数据库安全标准

国内标准，国际标准

- 数据库的访问控制

强制访问控制，自主访问控制

- 未来数据库安全技术